AI-BASED CUSTOMER FEEDBACK & EMOTION ANALYSIS SYSTEM

MINI PROJECT

Submitted by

SUDHARSHAN S - 61782324102195

VIGNESH KARTHIKEYAN M.R - 61782324102209

VIGNESHSARAVANAN M.S - 61782324102210

In partial fulfilment for the award of the degree

Of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

SONA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

SALEM – 636005

ANNA UNIVERSITY : CHENNAI – 600025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report **"AI-BASED CUSTOMER FEEDBACK & EMOTION ANALYSIS SYSTEM"** is the bonafide work of **SUDHARSHAN S, VIGNESH KARTHIKEYAN M.R and VIGNESHSARAVANAN M.S** who carried out the project work under my supervision.

SIGNATURE
Ms K. Theivani , M.E.,
ASSISTANT PROFESSORR,
SUPERVISOR
Department of CSE,
Sona College of Technology,
Salem – 636005.

SIGNATURE
Dr. B. SATHYABHAMA, M.TECH., Ph. D.
PROFESSOR,
HEAD OF THE DEPARTMENT
Department of CSE,
Sona College of Technology,
Salem – 636005.

Submitted for the Project Viva Voce Examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all who contributed to the successful completion of this project on **"AI-BASED CUSTOMER FEEDBACK & EMOTION ANALYSIS SYSTEM".**

I am deeply thankful to my project guide, Ms. K. Theivani, Assistant Professor, Department of Computer Science and Engineering, for her guidance, encouragement, and valuable suggestions throughout this work.

My sincere thanks to Dr. B. Sathiyabhama, Head of the Department of Computer Science and Engineering, for providing the facilities and support needed to complete this project. I am also thankful to the faculty members and staff of the department for their assistance.

I extend special thanks to my family and friends for their motivation and constant support during the project.

Finally, I acknowledge the open-source communities and researchers whose resources and publications on natural language processing, machine learning and sentiment analysis helped shape this work.

ABSTRACT

Businesses receive large volumes of customer feedback through reviews, surveys, emails and support channels. Reading this feedback manually is slow, inconsistent and hard to scale, so recurring complaints and changing customer satisfaction levels are often noticed too late. The AI-Based Customer Feedback & Emotion Analysis System is a web-based application that automates this analysis. It is developed using Python, Flask, Pandas, Scikit-learn and SQLite.

Users can enter feedback manually or upload CSV and TXT files for batch analysis. Using Natural Language Processing (NLP), TF-IDF text vectorization and Logistic Regression, the system predicts whether each piece of feedback is positive, neutral or negative. It also identifies potential emotions such as joy, anger, sadness, fear and surprise, and classifies feedback into categories such as product quality, delivery, customer service and payment.

A key feature is automated feedback classification and priority detection. The system assigns a high, medium or low priority to each entry so that complaints that need attention can be identified quickly. Analyzed feedback and predictions are stored in an SQLite database and presented through an interactive dashboard with sentiment and emotion charts, summary statistics and feedback history. The analyzed data can be exported as a CSV file for further analysis.

The main objective is to make customer feedback analysis more automated, accessible and efficient, helping businesses understand customer experiences, identify recurring issues and make better-informed decisions. The system relies only on open-source tools and a lightweight embedded database, so it can be deployed without commercial software licenses.

SDG addressed: Sustainable Development Goal 9 – Industry, Innovation and Infrastructure. Justification: SDG 9 promotes innovation and the adoption of technology, and Target 9.5 encourages enhancing technological capabilities of industrial sectors. The proposed system brings AI-based text analytics, which is usually available only to large organizations, within reach of small and medium businesses by using open-source tools. It also supports SDG 12 (Responsible Consumption and Production) by helping organizations detect product-quality and delivery problems earlier.

TABLE OF CONTENTS

Page No.

1. INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Need for the Proposed System 1
1.4 Objectives 1
1.5 Scope of the Project 2
1.6 Sustainable Development Goal 2
2. LITERATURE SURVEY 3
2.1 Papers that Contributed to the Problem Formulation 3
2.2 Research Issues / Problem Identified 4
2.2.1 Research Gap Addressed by the Proposed System 4
3. PROPOSED SYSTEM: AI-BASED CUSTOMER FEEDBACK & EMOTION ANALYSIS SYSTEM 5
3.1 Theoretical Background / Concepts / Methodology / Design / Modeling 5
3.1.1 Natural Language Processing 5
3.1.2 TF-IDF Text Vectorization 5
3.1.3 Logistic Regression Classifier 5
3.1.4 Emotion Detection 5
3.1.5 Category Classification 5
3.1.6 Priority Detection 6
3.1.7 System Architecture 6
3.1.8 Data Layer 7
3.1.9 NLP and Machine Learning Layer 7
3.2 Implementation Details 8
3.2.1 Frontend 8
3.2.2 Backend 8
3.2.3 Database 9
3.2.4 Feedback Input and Batch Upload 9
3.2.5 Sentiment Model 9
3.2.6 Emotion and Category Detection 9
3.2.7 Priority Detection 10
3.2.8 Dashboard and Export 10
3.2.9 Technology Stack 10
4. PERFORMANCE EVALUATION (RESULTS AND DISCUSSION) 11
4.1 Experimental Setup 11
4.2 Evaluation Parameters 11
4.3 Sentiment Classification Results 11
4.4 Emotion Detection Results 12
4.5 Category and Priority Analysis 13
4.6 Batch Processing and System Performance 13
4.7 Discussion 14

5. CONCLUSION AND FUTURE SCOPE.....15
5.1 Conclusion.....15
5.2 Future Scope.....15
REFERENCES.....16
ANNEXURE: CODE AND SCREENSHOTS.....17

LIST OF TABLES

Table 2.1 Comparison of Existing Sentiment and Emotion Analysis Approaches 3
Table 3.1 Data Layer Components 7
Table 3.2 NLP and ML Components Used in the System 7
Table 3.3 Priority Assignment Rules 10
Table 3.4 Proposed Technology Stack 10
Table 4.1 Evaluation Parameters 11
Table 4.2 Sentiment Classification Results 12
Table 4.3 Category and Priority Analysis 13
Table 4.4 Batch Processing Performance 13

LIST OF FIGURES

Figure 3.1 Overall Architecture of the Feedback & Emotion Analysis System 6
Figure 3.2 Data Flow in the Feedback Analysis System.....6
Figure 3.3 NLP and ML Processing Pipeline.....8
Figure 3.4 Dashboard Layout of the Application.....8
Figure 3.5 Feedback Analysis Workflow.....9
Figure 4.1 Sentiment Distribution.....12
Figure 4.2 Emotion Distribution 12
Figure 4.3 Priority Distribution by Category 13

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
TF-IDF	Term Frequency – Inverse Document Frequency
LR	Logistic Regression
CSV	Comma-Separated Values
TXT	Plain Text File
SQL	Structured Query Language
SQLite	Lightweight embedded SQL database engine
API	Application Programming Interface
UI	User Interface
CRUD	Create, Read, Update, Delete
CX	Customer Experience
F1	F1-Score (harmonic mean of precision and recall)
SDG	Sustainable Development Goal

CHAPTER 1
INTRODUCTION

1.1 Background

Customer feedback is one of the most direct indicators of how well a product or service meets expectations. Modern businesses collect it continuously through online reviews, post-purchase surveys, e-mails, chat transcripts and social media. This feedback contains not only an overall opinion but also emotions, specific complaints and suggestions for improvement.

Most of this information is unstructured text. Reading it manually does not scale and depends on the judgment of individual reviewers. Natural Language Processing (NLP) and machine learning make it possible to convert free text into structured information such as sentiment, emotion, topic and urgency, which can then be summarized, charted and acted upon.

1.2 Problem Statement

The problem addressed in this project is to build a web-based system that accepts customer feedback, either typed manually or uploaded in bulk as CSV or TXT files, and automatically determines customer satisfaction levels, emotions, complaint categories and the priority with which each entry should be handled. The system must store the analyzed results, present them through a dashboard, and allow them to be exported for further analysis.

1.3 Need for the Proposed System

Organizations that receive hundreds or thousands of feedback entries cannot read and classify each one in time. Important complaints may be buried among routine comments, and recurring problems in delivery, payment or customer service may stay unnoticed. Delayed responses to dissatisfied customers can lead to churn and reputational damage.

Existing enterprise sentiment-analysis platforms are often expensive, require cloud subscriptions or give only a positive/negative label without emotion, category or priority. A lightweight, open-source and locally deployable tool that provides all of these outputs in one dashboard is therefore needed, especially for small and medium businesses and student or research settings.

1.4 Objectives

- Develop a web application for entering feedback manually and uploading CSV and TXT files for batch analysis.
- Predict the sentiment of each feedback entry as positive, neutral or negative using TF-IDF and Logistic Regression.

- Identify potential emotions such as joy, anger, sadness, fear and surprise.
- Classify feedback into categories such as product quality, delivery, customer service and payment.
- Assign high, medium or low priority levels to help identify complaints that require attention.
- Store analyzed feedback and predictions in an SQLite database for history and later analysis.
- Provide an interactive dashboard with sentiment and emotion charts, summary statistics and feedback history.
- Allow analyzed feedback to be exported as a CSV file.
- Use open-source tools so that the system is reproducible without commercial licenses.

1.5 Scope of the Project

The system is intended for analyzing short customer feedback texts in English, such as product reviews, survey comments and support remarks. It supports single-entry analysis and batch analysis of CSV and TXT files, and provides a single-user web dashboard running on a Flask server with a local SQLite database.

This Phase-I report presents the problem formulation, literature survey, proposed architecture, methodology, implementation design and evaluation plan. Numerical experimental results will be produced during the implementation and testing phase, and the corresponding sections of Chapter 4 are therefore presented as an evaluation framework with result templates.

1.6 Sustainable Development Goal

The project addresses Sustainable Development Goal (SDG) 9, Industry, Innovation and Infrastructure, by making AI-based text analytics accessible through open-source tools and a lightweight architecture. It also supports SDG 12, Responsible Consumption and Production, by helping businesses detect product-quality and delivery issues early and respond to them.

CHAPTER 2
LITERATURE SURVEY

2.1 Papers that Contributed to the Problem Formulation

Sentiment analysis has been studied extensively. Pang and Lee [1] provide a comprehensive survey of opinion mining and sentiment analysis, covering the move from document-level polarity classification to finer-grained analysis. Liu [2] presents a detailed treatment of sentiment analysis as an NLP problem, including aspect-based analysis, and Medhat et al. [3] survey algorithms and applications and compare lexicon-based and machine-learning approaches.

Classical text-classification systems represent documents with term-weighting schemes. Salton and Buckley [4] describe term-weighting approaches that form the basis of TF-IDF, and the Scikit-learn library [5] provides an open and well-tested implementation of TF-IDF vectorization and Logistic Regression that is widely used for text-classification baselines.

Rule-based methods such as VADER [6] use a sentiment lexicon with heuristics for punctuation, capitalization and negation and work well on short informal text. For emotion analysis, Ekman [7] proposed a set of basic emotions that includes joy, anger, sadness, fear and surprise, and Mohammad and Turney [8] created a crowdsourced word–emotion association lexicon that is used for emotion detection. More recent work applies deep learning [9] and pre-trained transformer models such as BERT [10], which provide high accuracy but need greater computational resources and data.

These works shape the choice made in this project: a transparent, fast and reproducible TF-IDF plus Logistic Regression pipeline for sentiment, combined with emotion and category cues and a rule-based priority layer, packaged as an easy-to-use web dashboard.

Note: the works above are summarized from well-known literature; exact volume, page and DOI information should be verified from the original sources before final submission.

Table 2.1 compares the existing approaches and shows what each contributes to, or leaves open for, the proposed system.

Table 2.1 Comparison of Existing Sentiment and Emotion Analysis Approaches

Approach / Reference	Contribution	Limitation / Relevance to this Project
Survey of opinion mining and sentiment analysis [1][2][3]	Defines tasks, levels and methods of sentiment analysis.	Mostly theoretical; does not provide a ready dashboard with priority detection.

Approach / Reference	Contribution	Limitation / Relevance to this Project
TF-IDF term weighting with Scikit-learn classifiers [4][5]	Fast, interpretable text features and linear classifiers.	Needs labeled data; ignores word order and context.
Lexicon and rule-based analysis (VADER) [6]	Works without training; handles negation and intensity.	Limited for domain-specific words; polarity only, no category or priority.
Basic emotion models and emotion lexicons [7][8]	Provide emotion categories such as joy, anger, sadness, fear and surprise.	Lexicon coverage is limited; emotion alone does not indicate urgency.
Deep learning and transformer models [9][10]	Higher accuracy through contextual representations.	Require more data, computing power and deployment effort.

2.2 Research Issues / Problem Identified

The common gap is that existing work demonstrates the potential of sentiment analysis, but many solutions provide only a polarity label, require heavy infrastructure or are not packaged for non-technical users. The main issues identified are listed below.

- Feedback is unstructured and varies widely in length, spelling and style.
- Polarity alone does not explain why customers are dissatisfied; emotion and category information is also needed.
- Complaints that need urgent action are not automatically separated from routine comments.
- Advanced models need large datasets and computing resources that small organizations may not have.
- Many tools do not support batch uploads, persistent history and export in one workflow.
- Results are often not presented visually in a form that managers can use directly.

2.2.1 Research Gap Addressed by the Proposed System

The proposed system responds to these issues through a lightweight and reproducible TF-IDF plus Logistic Regression pipeline; combined sentiment, emotion and category analysis; automated priority detection for complaints; batch processing of CSV and TXT files; persistent storage in SQLite; an interactive dashboard; and CSV export of the analyzed data.

CHAPTER 3
PROPOSED SYSTEM: AI-BASED CUSTOMER FEEDBACK & EMOTION ANALYSIS SYSTEM

3.1 Theoretical Background / Concepts / Methodology / Design / Modeling

3.1.1 Natural Language Processing

NLP enables computers to process human language. In this project NLP is used to clean and normalize feedback text before it is converted into numerical features. Typical preprocessing steps are conversion to lower case, removal of punctuation, numbers and extra spaces, tokenization and removal of stop words. These steps reduce noise so that the learning algorithm can focus on meaningful words.

3.1.2 TF-IDF Text Vectorization

Term Frequency–Inverse Document Frequency (TF-IDF) represents each document as a vector in which every term receives a weight. The weight increases with the number of times a term appears in the document (term frequency) and decreases with the number of documents that contain the term (inverse document frequency). Common words therefore receive low weights, while distinctive words such as “late”, “refund” or “excellent” receive higher weights.

3.1.3 Logistic Regression Classifier

Logistic Regression is a linear model that estimates the probability of each class from the weighted sum of the input features. For three sentiment classes (positive, neutral, negative) a multinomial form with the softmax function is used, and the class with the highest probability is selected. It is fast to train, performs well on high-dimensional sparse TF-IDF features, and its coefficients can be inspected to see which words influence a prediction.

3.1.4 Emotion Detection

Emotion detection identifies the likely emotion expressed in feedback. Following the basic-emotion model [7], the system considers joy, anger, sadness, fear and surprise. Emotion cues are obtained from emotion-related words and expressions in the text, optionally supported by an emotion lexicon [8], and the most strongly indicated emotion is reported together with the sentiment.

3.1.5 Category Classification

Each feedback entry is assigned to a business category such as product quality, delivery, customer service or payment. Category-specific keywords and phrases (for example “damaged”, “late delivery”, “rude staff”, “refund” or “transaction failed”) are used to decide

the category, which allows managers to see which area of the business is generating the most complaints.

3.1.6 Priority Detection

The priority level is derived from the combination of predicted sentiment, strength of negative emotion such as anger or fear, the presence of urgency or complaint words, and the category. Strongly negative feedback with urgent or serious wording is marked high, other negative or mixed feedback is marked medium, and positive or neutral feedback is marked low. This helps the team to address critical complaints first.

3.1.7 System Architecture

The system follows a four-layer architecture, as shown in Figure 3.1: the presentation layer (web dashboard), the application layer (Flask routes and file handling), the NLP and machine-learning layer, and the data layer (SQLite database and saved model). Each layer communicates only with the adjacent layer, which keeps the design modular and easy to extend.

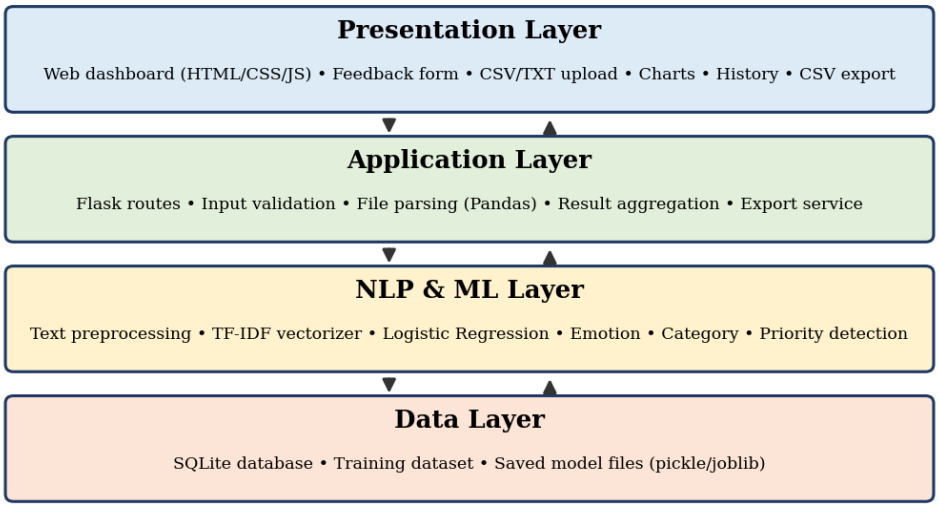


Figure 3.1 Overall Architecture of the Feedback & Emotion Analysis System

Figure 3.2 shows how data moves through the system. Feedback entered manually or read from an uploaded file is validated and parsed, preprocessed, and passed to the prediction components. The results are saved in SQLite and then displayed on the dashboard or exported as a CSV file.

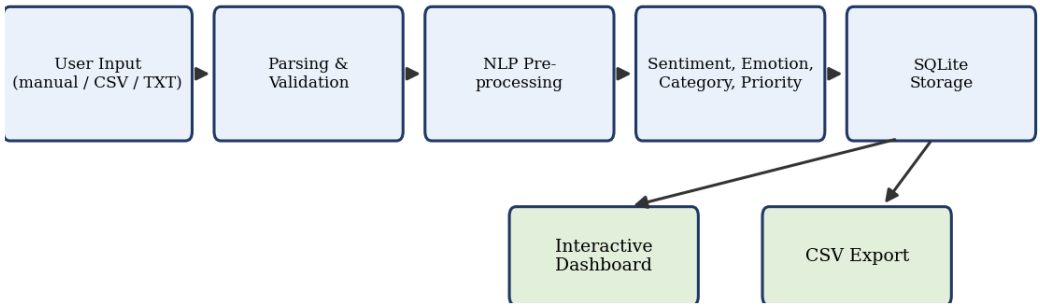


Figure 3.2 Data Flow in the Feedback Analysis System

3.1.8 Data Layer

The data layer holds the inputs and outputs summarized in Table 3.1. A labeled feedback dataset is used to train the sentiment model, and the trained model and vectorizer are saved so that they can be loaded when the application starts.

Table 3.1 Data Layer Components

Data	Source	Role in the System
Training dataset	Labeled customer reviews / feedback (CSV)	Used to train and test the TF-IDF and Logistic Regression model
Manual feedback	Text entered in the web form	Single-entry analysis
Uploaded feedback	CSV and TXT files uploaded by the user	Batch analysis of many entries
Saved model files	Serialized vectorizer and classifier	Loaded at start-up for prediction
Results database	SQLite tables	Stores text, sentiment, emotion, category, priority and timestamp
Exported file	CSV generated from the database	Further analysis in spreadsheets or BI tools

3.1.9 NLP and Machine Learning Layer

The NLP and ML layer contains the components listed in Table 3.2. Feedback text is cleaned, converted to TF-IDF vectors and passed to the classifier, while emotion, category and priority are determined from the text and the predicted sentiment, as shown in Figure 3.3.

Table 3.2 NLP and ML Components Used in the System

Task	Method	Main Inputs / Purpose
Text preprocessing	Lower-casing, cleaning, tokenization, stop-word removal	Normalizes raw feedback text
Feature extraction	TF-IDF vectorizer (Scikit-learn)	Converts text to weighted numeric vectors
Sentiment prediction	Logistic Regression	Predicts positive, neutral or negative
Emotion detection	Emotion cues / lexicon	Identifies joy, anger, sadness, fear or surprise
Category classification	Keyword and phrase based classification	Assigns product quality, delivery, customer service or payment

Task	Method	Main Inputs / Purpose
Priority detection	Rule-based scoring	Assigns high, medium or low priority

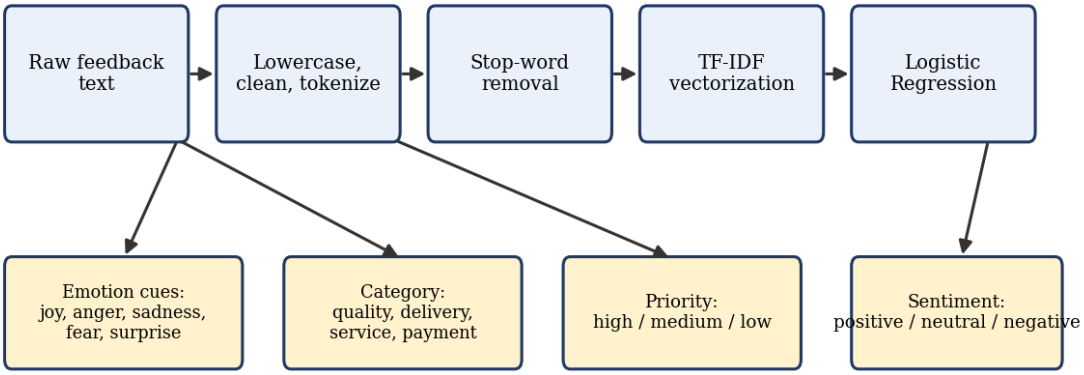


Figure 3.3 NLP and ML Processing Pipeline

3.2 Implementation Details

3.2.1 Frontend

The frontend is a web interface built with HTML, CSS and JavaScript and rendered through Flask templates. It contains a feedback entry form, a file upload control for CSV and TXT files, summary cards, sentiment and emotion charts, a searchable feedback history table and an export button. The layout of the dashboard is shown in Figure 3.4.

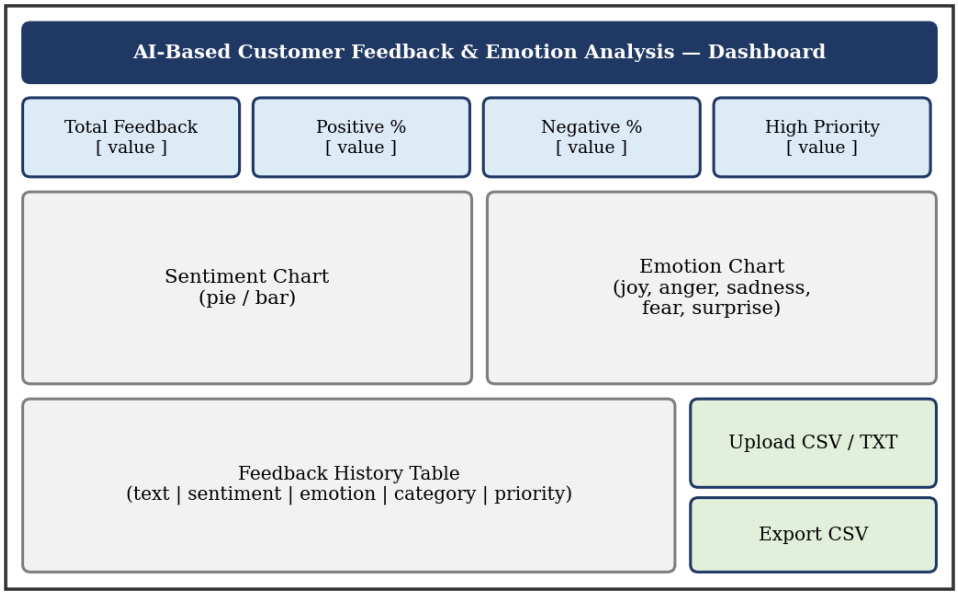


Figure 3.4 Dashboard Layout of the Application

3.2.2 Backend

Flask is used as the web framework because the NLP, machine-learning and data-handling libraries are all Python-based. Flask routes handle the home page, single feedback submission,

file upload, dashboard data, history retrieval and CSV export. Input validation checks for empty text, unsupported file types and missing columns before analysis.

3.2.3 Database

SQLite is used as an embedded database, so no separate database server is needed. A feedback table stores the original text, predicted sentiment, emotion, category, priority and the date and time of analysis. Dashboard statistics are produced by aggregate queries on this table, and the same table is used to generate the CSV export.

3.2.4 Feedback Input and Batch Upload

For manual entry, the text is analyzed immediately and the result is shown to the user. For batch analysis, a CSV file is read with Pandas and the feedback column is processed row by row; a TXT file is read with one feedback entry per line. All entries are analyzed in one pass and saved to the database. The workflow is shown in Figure 3.5.

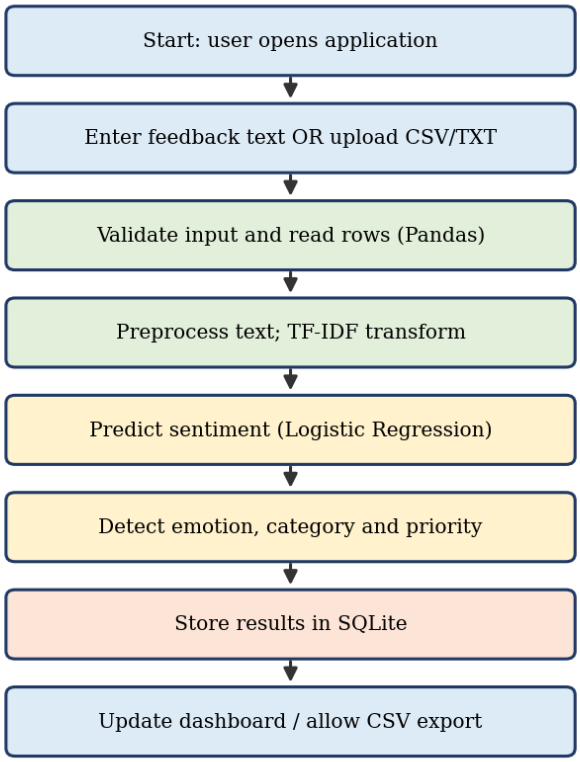


Figure 3.5 Feedback Analysis Workflow

3.2.5 Sentiment Model

The sentiment model is built as a Scikit-learn pipeline consisting of a TF-IDF vectorizer followed by a Logistic Regression classifier. The labeled dataset is divided into training and testing sets, and the trained pipeline is saved to disk and loaded when the application starts. The output is one of three classes: positive, neutral or negative.

3.2.6 Emotion and Category Detection

Emotion cues and category keywords are applied to the cleaned text. The detected emotion is one of joy, anger, sadness, fear or surprise, and the category is one of product quality, delivery, customer service or payment. If no clear cue is found, a neutral or general label is used so that every entry receives a result.

3.2.7 Priority Detection

Priority levels are assigned using the rules summarized in Table 3.3. The thresholds can be tuned after testing so that genuinely urgent complaints are identified without flooding the high-priority list.

Table 3.3 Priority Assignment Rules

Priority	Typical Conditions	Suggested Action
High	Negative sentiment with strong anger or fear, or urgent / serious complaint words	Respond and resolve immediately
Medium	Negative or mixed feedback without urgent wording	Review and respond within normal time
Low	Positive or neutral feedback, suggestions and general comments	Monitor and use for trend analysis

3.2.8 Dashboard and Export

The dashboard presents summary statistics such as total feedback, percentages of positive, neutral and negative entries and the number of high-priority complaints. Sentiment and emotion charts show the distribution of results, and the history table lists individual entries with their predictions. The export function writes the analyzed data from the database to a CSV file that can be opened in spreadsheet software.

3.2.9 Technology Stack

Table 3.4 lists the technologies selected for each part of the system. All core components are open source.

Table 3.4 Proposed Technology Stack

Layer	Technology
Programming language	Python 3
Web framework	Flask
Frontend	HTML, CSS, JavaScript with chart library (for example Chart.js)
Data handling	Pandas and NumPy
Machine learning	Scikit-learn (TF-IDF vectorizer, Logistic Regression)
NLP preprocessing	Python text processing / NLTK-style stop-word removal
Database	SQLite
File formats	CSV and TXT (input), CSV (export)

CHAPTER 4
PERFORMANCE EVALUATION (RESULTS AND DISCUSSION)

This report is a Phase-I (proposal and design) report. Measured prediction scores, processing times and user-study outcomes are not yet available. Numerical results are therefore not reported here. This chapter defines the evaluation setup, the parameters to be measured and the form in which results will be presented; the tables are templates and the figures are illustrative layouts that will be replaced with measured data after implementation.

4.1 Experimental Setup

The sentiment model will be trained and tested on a labeled customer-feedback dataset divided into training and testing portions (for example 80% and 20%). The complete application will be run locally on a standard laptop with Python, Flask, Pandas, Scikit-learn and SQLite. Test inputs will include manually entered feedback and CSV and TXT files of different sizes containing positive, neutral and negative comments covering the four categories.

4.2 Evaluation Parameters

The evaluation objectives are to measure the quality of sentiment classification; to check the correctness of emotion, category and priority assignments against manually labeled samples; to measure batch-processing time; and to verify the dashboard, storage and export functions. The metrics are listed in Table 4.1.

Table 4.1 Evaluation Parameters

Component	Metrics to Record
Sentiment classification	Accuracy, precision, recall, F1-score and confusion matrix
Emotion detection	Agreement with manually labeled samples; per-emotion precision and recall
Category classification	Accuracy and per-category precision and recall
Priority detection	Precision and recall for the high-priority class
Batch processing	Processing time versus number of feedback entries
Application functions	Upload success, database storage, dashboard update and export correctness

4.3 Sentiment Classification Results

The Logistic Regression model will be assessed with accuracy, precision, recall and F1-score for each sentiment class. Table 4.2 is the template in which the measurements will be recorded, and Figure 4.1 shows how the sentiment distribution of the analyzed feedback will be displayed. The bars in the figure are of equal height and are illustrative only.

Table 4.2 Sentiment Classification Results

Class	Precision	Recall	F1-Score
Positive	To be recorded	To be recorded	To be recorded
Neutral	To be recorded	To be recorded	To be recorded
Negative	To be recorded	To be recorded	To be recorded
Overall accuracy	To be recorded	—	—

Note: template; values to be recorded after experiments.

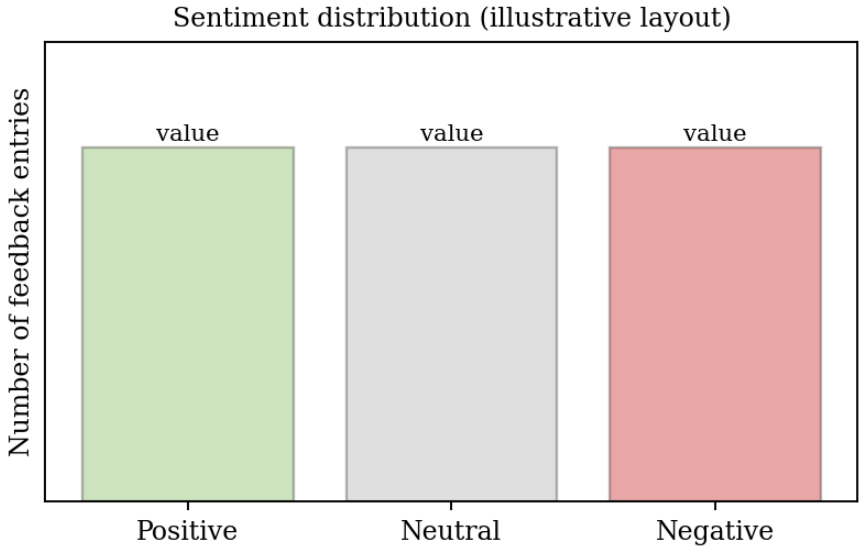


Figure 4.1 Sentiment Distribution

(Illustrative layout; not measured data)

4.4 Emotion Detection Results

The emotions identified for a manually labeled sample of feedback will be compared with the human labels. Figure 4.2 illustrates how the emotion distribution will be shown on the dashboard.

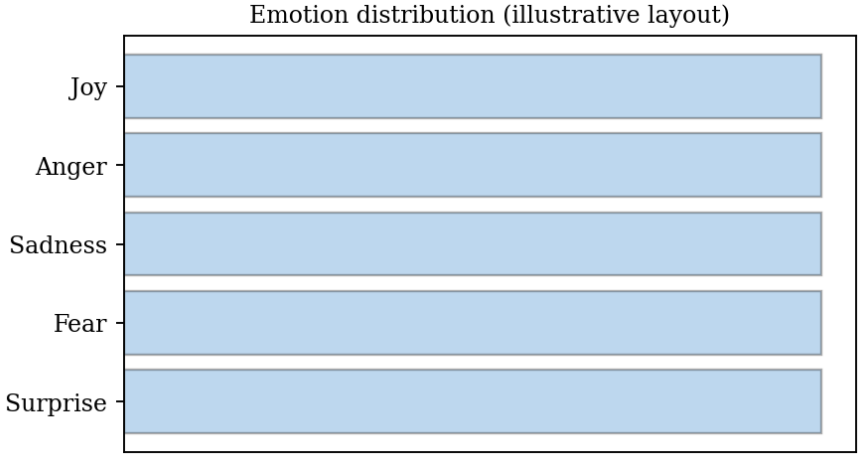


Figure 4.2 Emotion Distribution

(Illustrative layout; not measured data)

4.5 Category and Priority Analysis

For each category, the number of feedback entries and the number of high, medium and low priority entries will be recorded (Table 4.3) and visualized as shown in Figure 4.3. This analysis shows which areas of the business generate the most urgent complaints.

Table 4.3 Category and Priority Analysis

Category	Total	High	Medium	Low
Product quality	To be recorded	To be recorded	To be recorded	To be recorded
Delivery	To be recorded	To be recorded	To be recorded	To be recorded
Customer service	To be recorded	To be recorded	To be recorded	To be recorded
Payment	To be recorded	To be recorded	To be recorded	To be recorded

Note: template; values to be recorded after experiments.

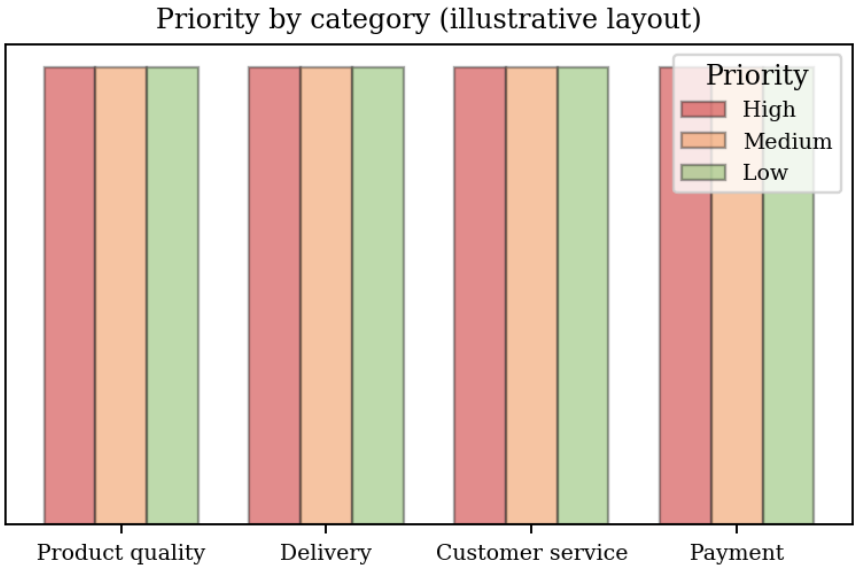


Figure 4.3 Priority Distribution by Category

(Illustrative layout; not measured data)

4.6 Batch Processing and System Performance

The time taken to analyze and store uploaded files containing different numbers of entries will be measured, together with the time for the dashboard to refresh and for the CSV export to complete (Table 4.4).

Table 4.4 Batch Processing Performance

Number of Entries	Analysis and Storage Time (s)	Dashboard Refresh Time (s)
100	To be recorded	To be recorded
1,000	To be recorded	To be recorded
10,000	To be recorded	To be recorded

Note: template; values to be recorded after experiments.

4.7 Discussion

The central hypothesis to be tested is that an automated pipeline combining TF-IDF, Logistic Regression, emotion cues, category classification and priority rules can analyze customer feedback quickly and with acceptable accuracy, and that the dashboard helps users to find complaints and recurring problems faster than manual reading. The evaluation will also examine the limits of the approach, such as sarcasm, mixed opinions and very short or ambiguous comments.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The AI-Based Customer Feedback & Emotion Analysis System proposes a practical way to convert large volumes of unstructured customer feedback into structured, actionable information. By combining NLP preprocessing, TF-IDF vectorization and Logistic Regression with emotion detection, category classification and priority assignment, it gives a single view of customer satisfaction, emotions and complaints.

The project addresses the gaps identified in the literature: polarity-only analysis, dependence on heavy infrastructure, lack of urgency detection and the absence of a ready-to-use dashboard with storage and export. By using open-source tools and an embedded database, it provides a reproducible foundation for businesses to understand customer experiences, identify recurring issues and make better-informed decisions.

5.2 Future Scope

- Train and compare stronger models such as Support Vector Machines, ensembles and transformer-based models.
- Replace rule-based emotion and category detection with supervised classifiers trained on labeled data.
- Add support for multiple languages and code-mixed text.
- Handle sarcasm, negation and mixed opinions with context-aware models.
- Integrate live feedback sources such as e-mail, social media and review-site APIs.
- Add user authentication, multi-user roles and a production database such as PostgreSQL.
- Provide trend analysis over time, keyword clouds and automatic alerts for high-priority complaints.
- Deploy the application on a cloud platform with containerization for wider access.

REFERENCES

1. B. Pang and L. Lee, “Opinion Mining and Sentiment Analysis,” Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.
2. B. Liu, Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers, 2012.
3. W. Medhat, A. Hassan and H. Korashy, “Sentiment analysis algorithms and applications: A survey,” Ain Shams Engineering Journal, vol. 5, no. 4, pp. 1093–1113, 2014.
4. G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.
5. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
6. C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text,” Proc. International AAAI Conference on Weblogs and Social Media (ICWSM), 2014.
7. P. Ekman, “An argument for basic emotions,” Cognition & Emotion, vol. 6, no. 3–4, pp. 169–200, 1992.
8. S. M. Mohammad and P. D. Turney, “Crowdsourcing a Word–Emotion Association Lexicon,” Computational Intelligence, vol. 29, no. 3, pp. 436–465, 2013.
9. L. Zhang, S. Wang and B. Liu, “Deep learning for sentiment analysis: A survey,” WIREs Data Mining and Knowledge Discovery, vol. 8, no. 4, 2018.
10. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proc. NAACL-HLT, 2019.

Note: bibliographic details of the above sources are to be verified from the original databases before final submission.

ANNEXURE: CODE AND SCREENSHOTS

Project: AI-Based Customer Feedback & Emotion Analysis System. The code below is a representative outline of the sentiment pipeline and Flask routes described in Chapter 3. It should be replaced or completed with the final project source code, and the application screenshots (dashboard, upload page, history table and exported CSV) should be inserted after implementation.

CODE:

model.py (sentiment pipeline)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline


# Starter training dataset for the FeedbackIQ project.
# Each entry contains:
# (feedback text, sentiment, emotion, category)

TRAINING_DATA = [
    ("Amazing product, excellent quality and fast delivery",
     "Positive", "Joy", "Product Quality"),
    ("I love this, it works perfectly",
     "Positive", "Joy", "Product Quality"),
    ("Very happy with the service, thank you",
     "Positive", "Joy", "Customer Service"),
    ("The package arrived early and everything was great",
     "Positive", "Joy", "Delivery"),
    ("Good value for money and easy to use",
     "Positive", "Joy", "Pricing"),
    ("The product is okay, nothing special",
     "Neutral", "Neutral", "Product Quality"),
    ("It is average and does what it says",
     "Neutral", "Neutral", "Product Quality"),
```

```
("I have a question about my payment",
 "Neutral", "Neutral", "Payment"),
("Can you tell me when my order will arrive",
 "Neutral", "Neutral", "Delivery"),
("I need help with my account",
 "Neutral", "Neutral", "Technical Issue"),
("Terrible product, it broke after one day",
 "Negative", "Anger", "Product Quality"),
("My order is late and I am very disappointed",
 "Negative", "Anger", "Delivery"),
("I was charged twice, fix my payment now",
 "Negative", "Anger", "Payment"),
("The app keeps crashing and nothing works",
 "Negative", "Anger", "Technical Issue"),
("Customer support was rude and unhelpful",
 "Negative", "Anger", "Customer Service"),
("I want a refund, this is unacceptable",
 "Negative", "Anger", "Refund / Return"),
("I am worried my personal details are not safe",
 "Negative", "Fear", "Technical Issue"),
("I feel sad because the gift arrived damaged",
 "Negative", "Sadness", "Delivery"),
("Wow, the upgrade is much better than expected",
 "Positive", "Surprise", "Product Quality"),
("The price is too expensive for this quality",
 "Negative", "Anger", "Pricing"),
("How do I return this item",
 "Neutral", "Neutral", "Refund / Return"),
("The refund has not arrived yet",
 "Negative", "Sadness", "Refund / Return"),
("Excellent support team solved my issue quickly",
 "Positive", "Joy", "Customer Service"),
("I cannot log in to my account",
 "Negative", "Anger", "Technical Issue"),
("Delivery was quick and the package was safe",
```

```
    "Positive", "Joy", "Delivery"),
    ("The product is defective and damaged",
    "Negative", "Anger", "Product Quality"),
    ("I am afraid I will lose my money",
    "Negative", "Fear", "Payment"),
    ("Unexpected charge appeared on my bill",
    "Negative", "Surprise", "Payment"),
    ("The website is simple and easy to navigate",
    "Positive", "Joy", "Technical Issue"),
    ("Please cancel my order and return my money",
    "Negative", "Anger", "Refund / Return"),

    # Additional examples to improve common complaint recognition.
    ("The quality is very poor", "Negative", "Anger", "Product Quality"),
    ("The item is bad quality", "Negative", "Anger", "Product Quality"),
    ("I am unhappy with the product", "Negative", "Sadness", "Product Quality"),
    ("The product is wonderful", "Positive", "Joy", "Product Quality"),
    ("I am extremely satisfied with my purchase",
    "Positive", "Joy", "Product Quality"),
    ("The delivery took too long",
    "Negative", "Anger", "Delivery"),
    ("My package never arrived",
    "Negative", "Sadness", "Delivery"),
    ("The support team was excellent",
    "Positive", "Joy", "Customer Service"),
    ("I am scared my account has been hacked",
    "Negative", "Fear", "Technical Issue"),
    ("The unexpected replacement was amazing",
    "Positive", "Surprise", "Product Quality"),
]


TEXTS = [item[0] for item in TRAINING_DATA]
SENTIMENTS = [item[1] for item in TRAINING_DATA]
EMOTIONS = [item[2] for item in TRAINING_DATA]
```

```
CATEGORIES = [item[3] for item in TRAINING_DATA]


def create_model(labels):
    """Build and train a TF-IDF and Logistic Regression pipeline."""
    pipeline = Pipeline([
        (
            "tfidf",
            TfidfVectorizer(
                lowercase=True,
                ngram_range=(1, 2),
                sublinear_tf=True
            )
        ),
        (
            "classifier",
            LogisticRegression(
                max_iter=1000,
                class_weight="balanced",
                random_state=42
            )
        )
    ])

    pipeline.fit(TEXTS, labels)
    return pipeline


# Train the three prediction models when this module is imported.
sentiment_model = create_model(SENTIMENTS)
emotion_model = create_model(EMOTIONS)
category_model = create_model(CATEGORIES)


def predict_sentiment(text):
```

```
    """Predict Positive, Neutral, or Negative sentiment."""
    return sentiment_model.predict([text])[0]


def predict_emotion(text):
    """Predict the most likely emotion label."""
    return emotion_model.predict([text])[0]


def predict_category(text):
    """Predict the customer feedback category."""
    return category_model.predict([text])[0]


def analyze_feedback(text):
    """Return sentiment, emotion, and category predictions."""
    if not isinstance(text, str) or not text.strip():
        raise ValueError("Feedback text cannot be empty.")

    cleaned_text = " ".join(text.split())

    return {
        "feedback": cleaned_text,
        "sentiment": predict_sentiment(cleaned_text),
        "emotion": predict_emotion(cleaned_text),
        "category": predict_category(cleaned_text)
    }


if __name__ == "__main__":
    print("FeedbackIQ machine-learning model test")
    print("-" * 40)

    samples = [
        "The product is bad quality",
```

```
    "Amazing product and excellent service",
    "I am worried about my payment",
    "The delivery was average"
]

for sample in samples:
    result = analyze_feedback(sample)
    print("\nFeedback:", result["feedback"])
    print("Sentiment:", result["sentiment"])
    print("Emotion:", result["emotion"])
    print("Category:", result["category"])
```

app.py (Flask routes – outline)

```
from flask import (
    Flask, render_template, request, redirect,
    url_for, flash, send_file
)
from model import analyze_feedback
import pandas as pd
import sqlite3
import io
import os
from datetime import datetime

app = Flask(__name__)
app.secret_key = "feedbackiq-demo-secret-key"

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "feedback.db")
ALLOWED_EXTENSIONS = {"csv", "txt"}


def init_db():
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS feedback (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                feedback TEXT NOT NULL,
                sentiment TEXT NOT NULL,
                emotion TEXT NOT NULL,
                category TEXT NOT NULL,
                priority TEXT NOT NULL,
                created_at TEXT NOT NULL
            )
        """)


init_db()


def predict_feedback(text):
    prediction = analyze_feedback(str(text))

    cleaned = prediction["feedback"]
    sentiment = prediction["sentiment"]
    emotion = prediction["emotion"]
    category = prediction["category"]

    lower_text = cleaned.lower()

    urgent_words = [
```

```
        "urgent", "immediately", "asap", "fraud",
        "unsafe", "charged twice", "cannot access",
        "can't access", "not arrived", "never arrived",
        "refund", "broken", "damaged", "scam",
        "cancel", "emergency", "defective"
    ]

    if sentiment == "Negative" and (
        emotion in {"Anger", "Fear", "Sadness"}
        or any(word in lower_text for word in urgent_words)
    ):
        priority = "High"
    elif sentiment == "Negative":
        priority = "Medium"
    else:
        priority = "Low"

    return {
        "feedback": cleaned,
        "sentiment": sentiment,
        "emotion": emotion,
        "category": category,
        "priority": priority,
        "created_at": datetime.now().strftime(
            "%Y-%m-%d %H:%M:%S"
        )
    }


def save_result(result):
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute(
            """
            INSERT INTO feedback
            (feedback, sentiment, emotion, category, priority, created_at)
            VALUES (?, ?, ?, ?, ?, ?)
            """,
            (
                result["feedback"],
                result["sentiment"],
                result["emotion"],
                result["category"],
                result["priority"],
                result["created_at"]
            )
        )


def get_results(search_text="", sentiment_filter="", priority_filter=""):
    query = "SELECT * FROM feedback WHERE 1=1"
    params = []
```

```
    if search_text:
        query += " AND feedback LIKE ?"
        params.append(f"%{search_text}%")

    if sentiment_filter:
        query += " AND sentiment = ?"
        params.append(sentiment_filter)

    if priority_filter:
        query += " AND priority = ?"
        params.append(priority_filter)

    query += " ORDER BY id DESC"

    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(query, params).fetchall()
        return [dict(row) for row in rows]


def parse_upload(uploaded_file):
    filename = uploaded_file.filename or ""

    extension = (
        filename.rsplit(".", 1)[-1].lower()
        if "." in filename else ""
    )

    if extension not in ALLOWED_EXTENSIONS:
        raise ValueError("Please upload a CSV or TXT file.")

    if extension == "txt":
        content = uploaded_file.read().decode(
            "utf-8-sig", errors="replace"
        )
        return [
            line.strip()
            for line in content.splitlines()
            if line.strip()
        ]

    dataframe = pd.read_csv(uploaded_file)

    if dataframe.empty:
        return []

    possible_columns = [
        "feedback", "review", "comment",
        "text", "customer_feedback"
    ]
```

```
    column_lookup = {
        str(column).strip().lower(): column
        for column in dataframe.columns
    }

    selected_column = next(
        (
            column_lookup[name]
            for name in possible_columns
            if name in column_lookup
        ),
        dataframe.columns[0]
    )

    return (
        dataframe[selected_column]
        .dropna()
        .astype(str)
        .tolist()
    )


def get_summary(results):
    return {
        "total": len(results),
        "high": sum(
            row["priority"] == "High" for row in results
        ),
        "medium": sum(
            row["priority"] == "Medium" for row in results
        ),
        "low": sum(
            row["priority"] == "Low" for row in results
        ),
        "positive": sum(
            row["sentiment"] == "Positive" for row in results
        ),
        "neutral": sum(
            row["sentiment"] == "Neutral" for row in results
        ),
        "negative": sum(
            row["sentiment"] == "Negative" for row in results
        )
    }


def get_chart_data(results):
    sentiment_counts = {
        "Positive": 0,
        "Neutral": 0,
```

```
        "Negative": 0
    }

    emotion_counts = {}
    category_counts = {}

    for row in results:
        sentiment = row["sentiment"]
        emotion = row["emotion"]
        category = row["category"]

        if sentiment in sentiment_counts:
            sentiment_counts[sentiment] += 1

        emotion_counts[emotion] = (
            emotion_counts.get(emotion, 0) + 1
        )

        category_counts[category] = (
            category_counts.get(category, 0) + 1
        )

    return sentiment_counts, emotion_counts, category_counts


@app.route("/")
@app.route("/dashboard")
def index():
    results = get_results()
    summary = get_summary(results)

    sentiment_counts, emotion_counts, category_counts = (
        get_chart_data(results)
    )

    return render_template(
        "index.html",
        page="dashboard",
        results=results,
        summary=summary,
        sentiment_counts=sentiment_counts,
        emotion_counts=emotion_counts,
        category_counts=category_counts
    )


@app.route("/analyze", methods=["GET", "POST"])
def analyze():
    result = None

    if request.method == "POST":
```

```
        feedback_text = request.form.get("feedback", "").strip()

        try:
            result = predict_feedback(feedback_text)
            save_result(result)
            flash("Feedback analyzed and saved successfully.", "success")

        except ValueError as error:
            flash(str(error), "error")

    return render_template(
        "analyze.html",
        page="analyze",
        result=result
    )


@app.route("/upload", methods=["GET", "POST"])
def upload():
    if request.method == "POST":
        uploaded_file = request.files.get("file")

        if not uploaded_file or not uploaded_file.filename:
            flash("Please select a CSV or TXT file.", "error")
            return redirect(url_for("upload"))

        try:
            rows = parse_upload(uploaded_file)

            if not rows:
                flash("The uploaded file contains no feedback.", "error")
                return redirect(url_for("upload"))

            count = 0
            failed = 0

            for feedback_text in rows:
                try:
                    result = predict_feedback(feedback_text)
                    save_result(result)
                    count += 1
                except ValueError:
                    failed += 1

            if count:
                flash(
                    f"Successfully analyzed {count} feedback entries. "
                    f"Skipped {failed} invalid entries.",
                    "success"
                )
            else:
```

```
                flash("No valid feedback entries were processed.", "error")

            except Exception as error:
                app.logger.exception("Upload processing failed")
                flash(f"Could not process the file: {error}", "error")

            return redirect(url_for("upload"))

    return render_template("upload.html", page="upload")


@app.route("/history")
def history():
    search_text = request.args.get("search", "").strip()
    sentiment_filter = request.args.get("sentiment", "").strip()
    priority_filter = request.args.get("priority", "").strip()

    valid_sentiments = {"Positive", "Neutral", "Negative"}
    valid_priorities = {"High", "Medium", "Low"}

    if sentiment_filter not in valid_sentiments:
        sentiment_filter = ""

    if priority_filter not in valid_priorities:
        priority_filter = ""

    results = get_results(
        search_text,
        sentiment_filter,
        priority_filter
    )

    return render_template(
        "history.html",
        page="history",
        results=results,
        search=search_text,
        selected_sentiment=sentiment_filter,
        selected_priority=priority_filter
    )


@app.route("/about")
def about():
    return render_template("about.html", page="about")


@app.route("/export")
def export_csv():
    results = get_results()
```

```
    columns = [
        "id", "feedback", "sentiment", "emotion",
        "category", "priority", "created_at"
    ]

    dataframe = pd.DataFrame(results, columns=columns)
    output = io.BytesIO()

    dataframe.to_csv(
        output,
        index=False,
        encoding="utf-8-sig"
    )

    output.seek(0)

    return send_file(
        output,
        mimetype="text/csv",
        as_attachment=True,
        download_name="feedback_analysis.csv"
    )


@app.route("/clear", methods=["POST"])
def clear_results():
    with sqlite3.connect(DB_PATH) as conn:
        conn.execute("DELETE FROM feedback")

    flash("Feedback history cleared successfully.", "success")

    return redirect(url_for("index"))


@app.errorhandler(404)
def page_not_found(error):
    return (
        "<h1>404 - Page not found</h1>"
        "<p>The requested page does not exist.</p>"
        '<a href="/">Return to Dashboard</a>',
        404
    )


if __name__ == "__main__":
    app.run(debug=True)
```

SCREENSHOTS:

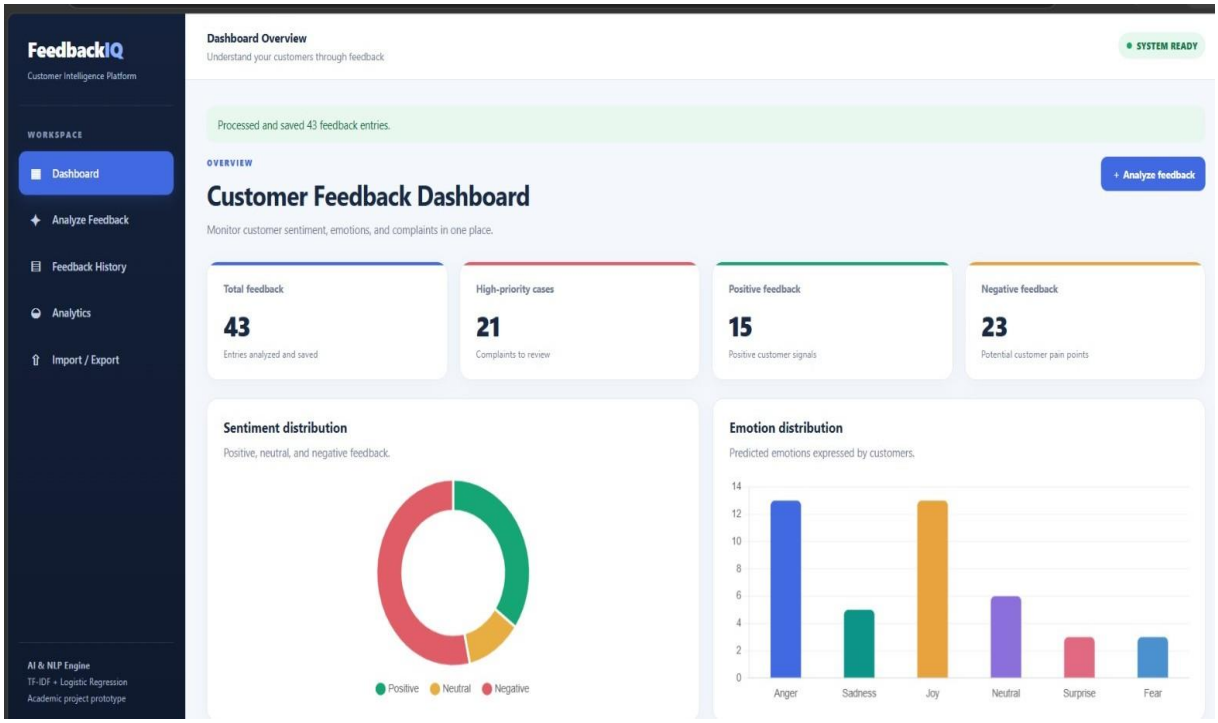


Figure A.1: Dashboard with the history of feedback analysis and its sentiment distribution and emotion distribution.

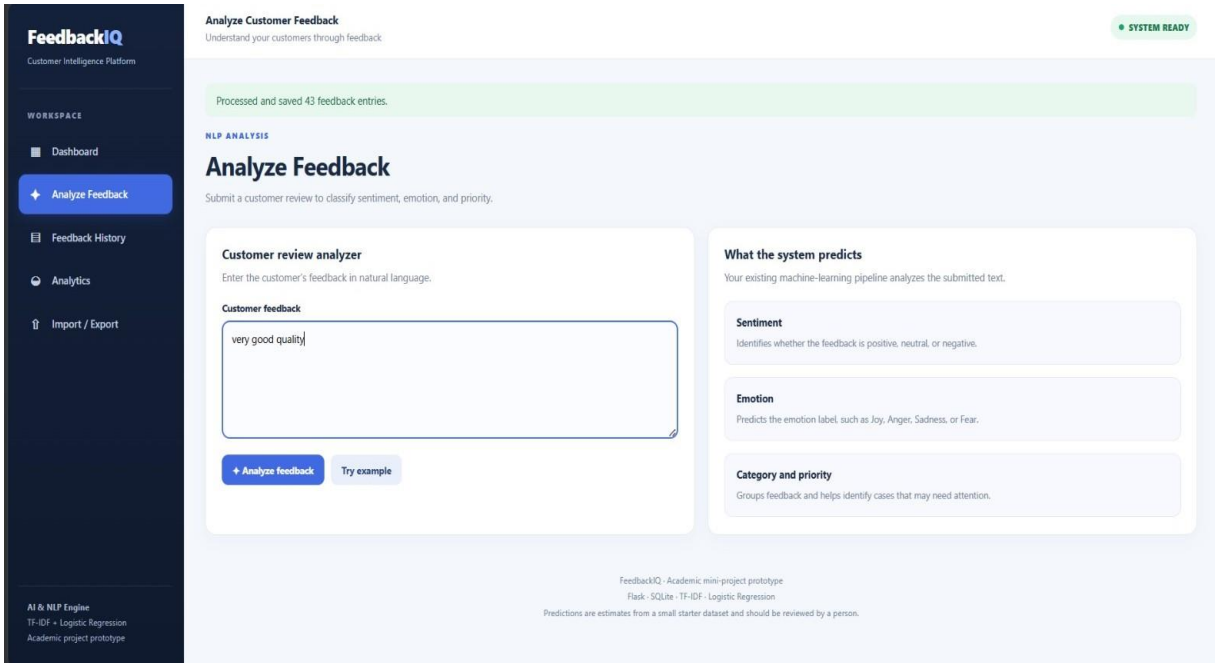
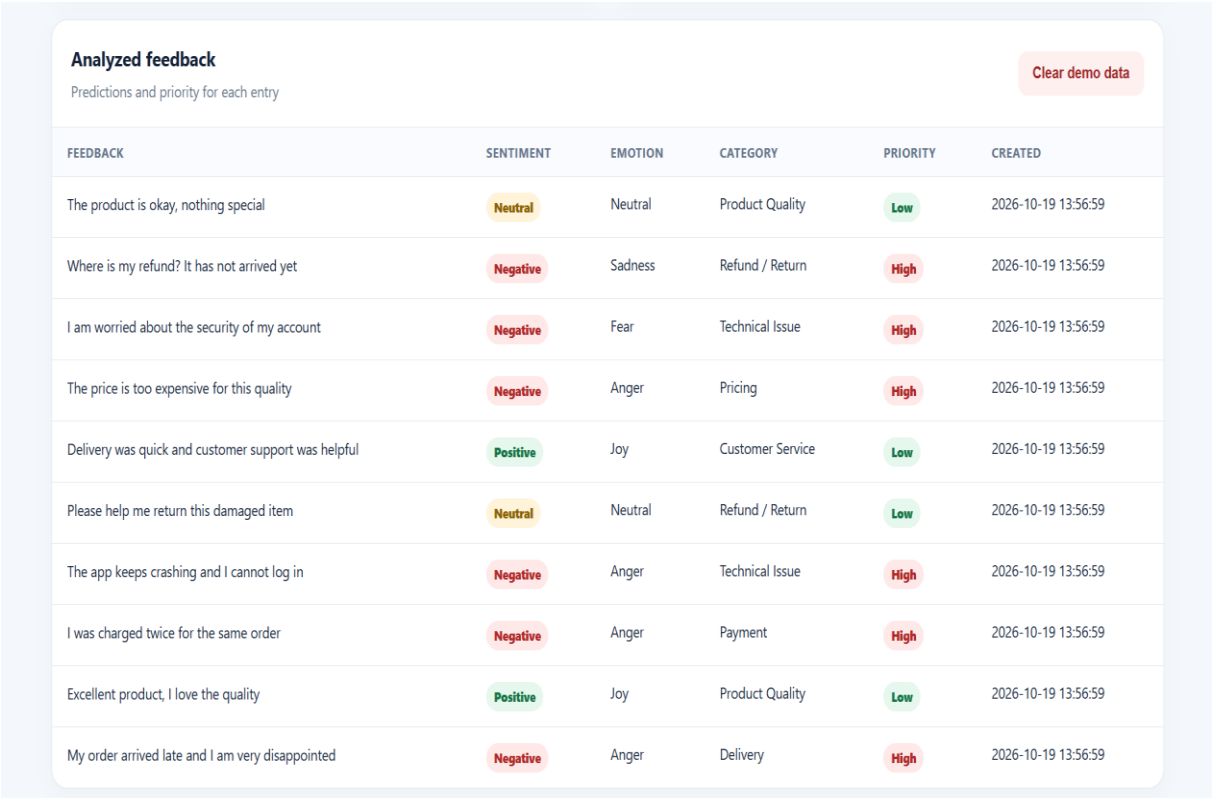


Figure A.2: This page analyses the feedback given by the user in text format or in .csv file



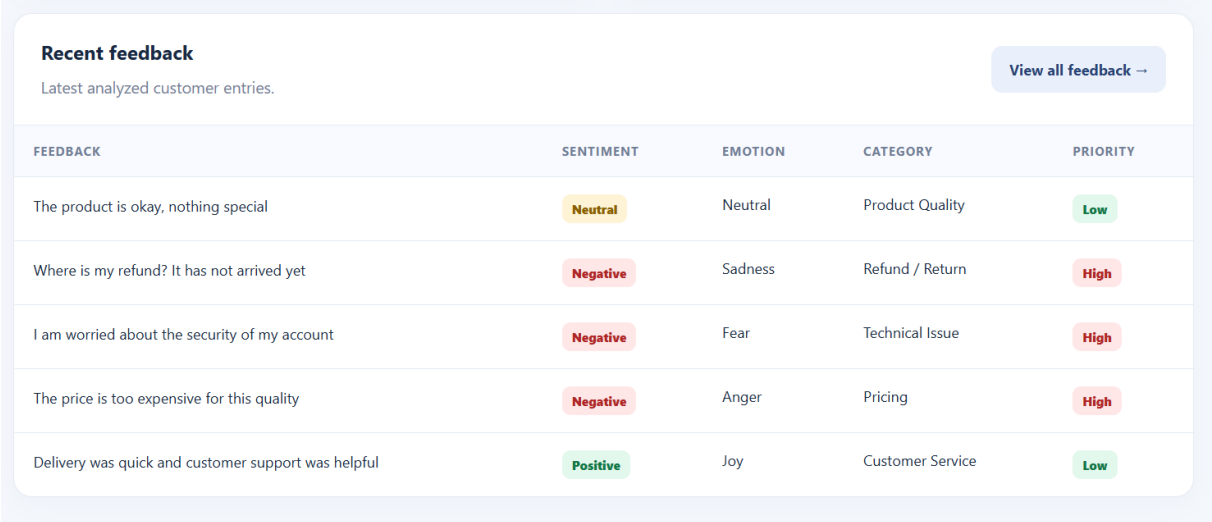
Analyzed feedback

Predictions and priority for each entry

Clear demo data

FEEDBACK	SENTIMENT	EMOTION	CATEGORY	PRIORITY	CREATED
The product is okay, nothing special	Neutral	Neutral	Product Quality	Low	2026-10-19 13:56:59
Where is my refund? It has not arrived yet	Negative	Sadness	Refund / Return	High	2026-10-19 13:56:59
I am worried about the security of my account	Negative	Fear	Technical Issue	High	2026-10-19 13:56:59
The price is too expensive for this quality	Negative	Anger	Pricing	High	2026-10-19 13:56:59
Delivery was quick and customer support was helpful	Positive	Joy	Customer Service	Low	2026-10-19 13:56:59
Please help me return this damaged item	Neutral	Neutral	Refund / Return	Low	2026-10-19 13:56:59
The app keeps crashing and I cannot log in	Negative	Anger	Technical Issue	High	2026-10-19 13:56:59
I was charged twice for the same order	Negative	Anger	Payment	High	2026-10-19 13:56:59
Excellent product, I love the quality	Positive	Joy	Product Quality	Low	2026-10-19 13:56:59
My order arrived late and I am very disappointed	Negative	Anger	Delivery	High	2026-10-19 13:56:59

Figure A.3: Analyzed feedback shows the sentiment, emotion and category of the feedback using TF-IDF and priority of the review using Logistic Regression



Recent feedback

Latest analyzed customer entries.

View all feedback →

FEEDBACK	SENTIMENT	EMOTION	CATEGORY	PRIORITY
The product is okay, nothing special	Neutral	Neutral	Product Quality	Low
Where is my refund? It has not arrived yet	Negative	Sadness	Refund / Return	High
I am worried about the security of my account	Negative	Fear	Technical Issue	High
The price is too expensive for this quality	Negative	Anger	Pricing	High
Delivery was quick and customer support was helpful	Positive	Joy	Customer Service	Low

Figure A.4: Now the feedback will be moved to the recent feedback panel and further can be exported into an another .csv file

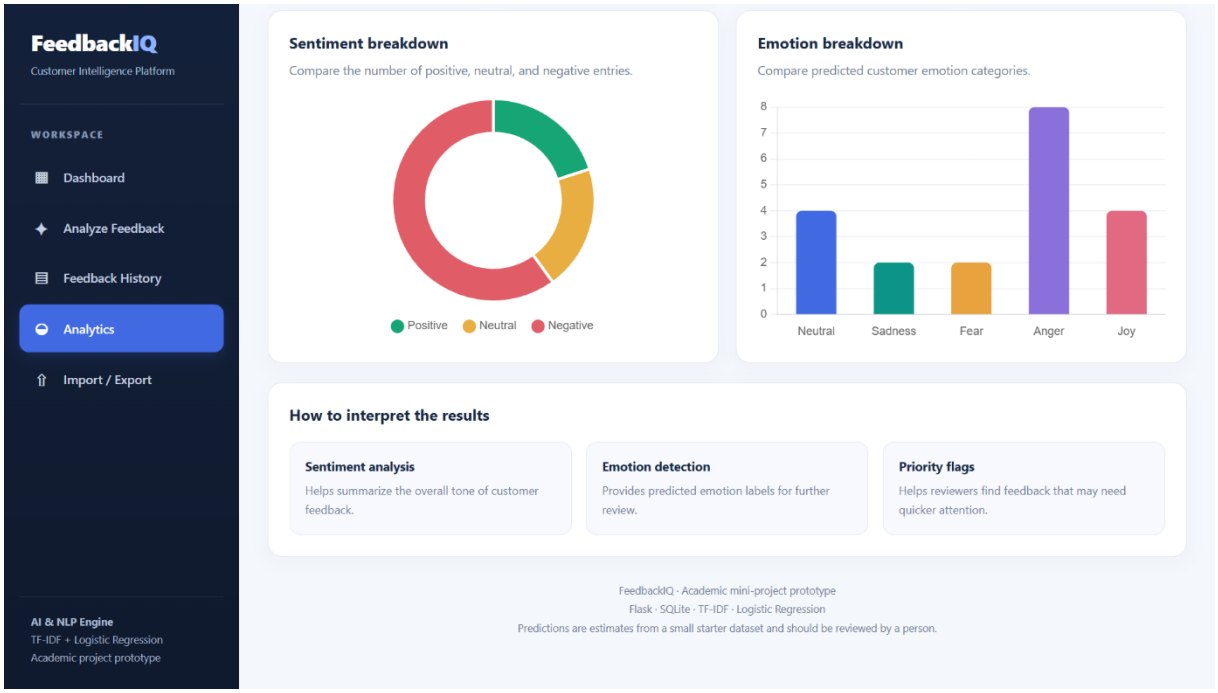


Figure A.5: FeedbackIQ visualizes customer sentiment and predicted emotions through charts, helping users understand feedback patterns and identify complaints that may require urgent attention.

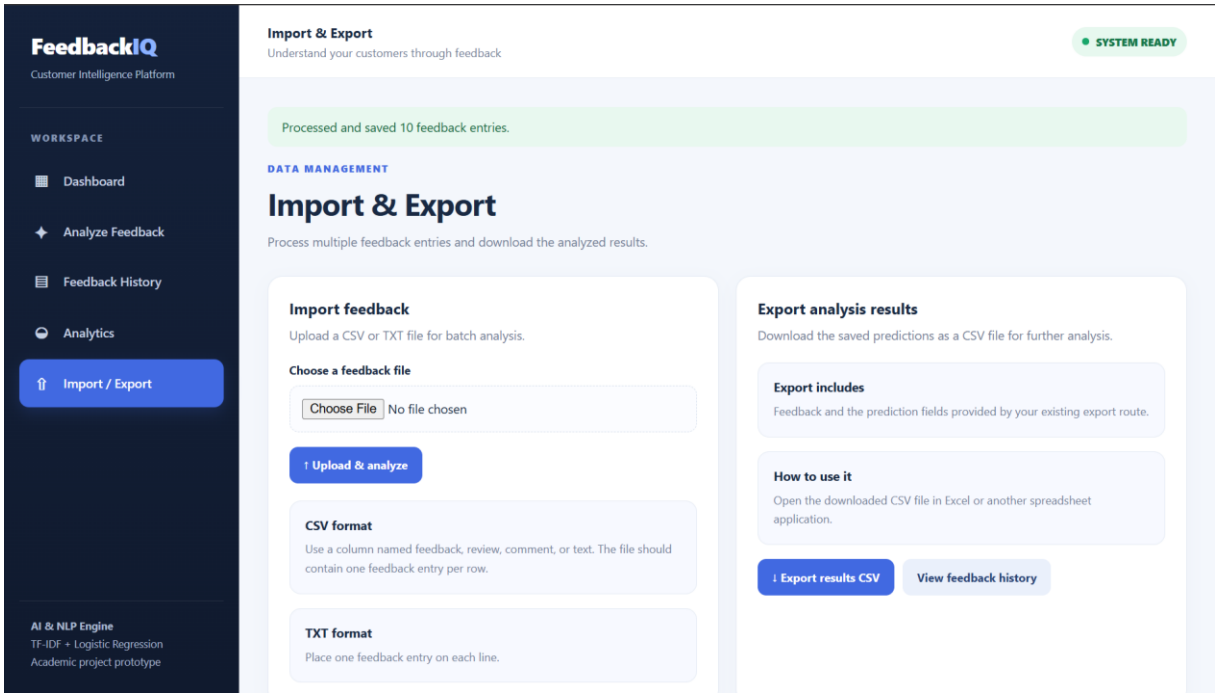


Figure A.6: All types of file an be uploaded for review analysis and the analysed result can be exported as a .csv file

SOURCE CODE AND API INVENTORY

The complete source code for the FeedbackIQ — AI-Based Customer Feedback and Emotion Analysis System is included in the project folder. Principal implementation files are listed below:

- app.py — Flask application, routing, feedback analysis, file uploads, dashboard rendering, CSV export, and database operations.
- templates/index.html — Main dashboard interface for entering feedback, uploading CSV/TXT files, viewing analysis results, and visualizing charts.
- feedback.db — SQLite database used to store analyzed customer feedback, predicted sentiment, emotion, category, priority, and timestamps.
- requirements.txt — Python dependencies required to run the application, including Flask, scikit-learn, pandas, and NumPy
- venv/ — Python virtual environment containing the project's installed dependencies.

Machine Learning and NLP Implementation

The system uses Natural Language Processing (NLP) and machine learning to analyze customer feedback.

- TF-IDF Vectorizer — Converts customer feedback text into numerical features for machine learning.
- Logistic Regression — Predicts sentiment, emotion, and feedback category using models trained on a small starter dataset.
- Rule-based emotion correction — Uses selected keywords and phrases to refine emotion predictions for common expressions.
- Priority classification — Assigns High, Medium, or Low priority based on predicted sentiment, emotion, and selected urgency-related keywords.

Application Features and Routes

The principal application routes are listed below:

- GET / — Displays the dashboard, feedback submission form, summary metrics, charts, and feedback history.
- POST / — Processes manually entered feedback or uploaded CSV/TXT files, depending on the submitted action.
- GET /export — Exports analyzed feedback records as a downloadable CSV file.
- POST /clear — Clears the stored feedback history for demonstration purposes.

Data Storage and File Processing

The application uses SQLite for persistent storage of analyzed feedback. Users can submit feedback individually or upload CSV and TXT files for batch processing. The CSV parser searches for a column named feedback, review, comment, text, or customer_feedback, and falls back to the first column when none of these is present. The export function generates a CSV file containing the stored analysis results.

Validation and Limitations

The application can be tested by running py app.py from the project directory and opening http://127.0.0.1:5000 in a browser. Validation should include manual feedback submission, CSV/TXT uploads, dashboard metric updates, chart rendering, CSV export, and feedback-history clearing.